NUTRIVIDA · DSY1104 DESARROLLO FULLSTACK II · EP2

Documento de cobertura del testing

Frontend React + Vite de la Clínica Nutricional NutriVida (Temuco). Entorno de pruebas, registro de las 12 pruebas oficiales, mocks y spies, métricas de cobertura y su análisis.

396
pruebas aprobadas en 62 archivos, 0 fallidas

12 / 12
pruebas oficiales aprobadas (27 casos)

94,69 %
de líneas cubiertas

83,30 %
de ramas cubiertas, la métrica más baja

Identificación

Asignatura	DSY1104 Desarrollo Fullstack II, 2026-2
Evaluación	Evaluación Parcial 2, situación evaluativa 1 (proyecto frontend)
Caso	Forma D, Clínica Nutricional NutriVida, Temuco
Sección y grupo	DSY1104-_____ · Grupo G_____
Repositorio	`github.com/Sandy-Yera/NutriVida_Proyect`, rama `main`, tag `ep2-entrega`
Medición	18-10-2026, con `npm run test:coverage` en `frontend/`, sobre `main` con las tres ramas integradas
Responsable	Gabriel (tarea G-11). Cada integrante aportó las filas de sus 4 pruebas.

Integrantes

Gabriel	Acceso y administración: registro, login, roles, panel admin de usuarios, dashboard y reportes
Simon	Citas: agendas, solicitud de cita en 4 pasos, pago simulado, reprogramación y administración de citas
Catalina	Sitio público y ficha clínica: catálogo de servicios, mapa, contacto, ficha, mediciones y progreso

1. Entorno de pruebas

Las pruebas viven junto al archivo que prueban, con el mismo nombre y la extensión `.test.js` o `.test.jsx`. Todo se ejecuta dentro de `frontend/`.

Herramienta	Versión	Para qué se usa
Vitest	`4.1.11`	Ejecuta las pruebas y entrega `describe`, `it`, `expect` y `vi` (mocks, spies y reloj simulado).
@vitest/coverage-v8	`4.1.11`	Mide la cobertura con el contador de V8, el motor de JavaScript de Node, y genera el informe de texto y el HTML.
@testing-library/react	`16.3.3`	Renderiza componentes y los consulta como lo haría una persona: por rol, etiqueta o texto visible.
@testing-library/dom	`10.4.2`	Base de las consultas (`getByRole`, `findByText`) y de `fireEvent`.
@testing-library/jest-dom	`6.9.1`	Agrega comparadores del DOM: `toBeInTheDocument`, `toHaveTextContent`, `toHaveAttribute`.
jsdom	`26.1.0`	Simula el navegador dentro de Node: DOM, eventos, `localStorage` y `sessionStorage`. No calcula diseño ni dibuja.
React / Vite	`19.3.0 / 8.3.4`	La aplicación probada y el empaquetador que Vitest reutiliza para transformar JSX.

vitest.config.js

```
export default defineConfig({
  plugins: [react()],
  test: {
    environment: "jsdom",
    globals: true,
    setupFiles: "./src/test/setup.js",
    css: false,
    coverage: {
      provider: "v8",
      reporter: ["text", "html"],
      include: ["src/**/*.{js,jsx}"],
      exclude: ["src/main.jsx", "src/test/**",
"src/data/semillas/**", "src/**/*.test.{js,jsx}"],
    },
  },
});
```

src/test/setup.js

```
import "@testing-library/jest-dom/vitest";
import { cleanup } from "@testing-library/react";
import { afterEach } from "vitest";

afterEach(() => {
  cleanup();
  localStorage.clear();
});
```

- **environment y globals**: un DOM falso para renderizar, y `expect` sin importarlo en cada archivo.
- **css: false**: no se procesan estilos; por eso el diseño responsivo se revisa en el navegador.
- **afterEach**: desmonta lo renderizado y limpia `localStorage`, así ninguna prueba hereda datos de otra.
- **coverage.exclude**: no mide el punto de entrada, las propias pruebas ni las semillas, que son solo datos.

Scripts

Comando	Ejecuta	Para qué
`npm test`	`vitest run`	Ejecuta las 396 pruebas una vez. Es el que se usa antes de cada commit.
`npm run test:watch`	`vitest`	Modo observación: repite las pruebas afectadas al guardar un archivo.
`npm run test:coverage`	`vitest run --coverage`	Ejecuta todo y mide la cobertura. Deja el informe en `coverage/index.html`.
`npx vitest run -t "[GSC]-T[1-4]"`	`filtro por nombre`	Ejecuta solo las 12 pruebas oficiales (27 casos), porque su ID va en el `describe` o en el `it`.

2. Registro de las 12 pruebas

La rúbrica pide 10 pruebas; el plan define 12, cuatro por integrante, para cubrir los cuatro tipos que pide la guía de testing. Cada ID va en el nombre del `describe` o del `it`, por eso el filtro `-t "[GSC]-T[1-4]"` las encuentra todas. Resultado del 18-10-2026: 27 casos aprobados de 27.

ID	Archivo	Comportamiento que verifica	Tipo	Mock o spy	Casos
G-T1 Gabriel	`src/utils/validaciones.test.js`	`validarRut` acepta un RUT con dígito verificador correcto (módulo 11), rechaza uno con el dígito cambiado y acepta el RUT escrito sin puntos ni guion y con *k* minúscula.	Lógica	—	3 ✓
G-T2 Gabriel	`src/utils/validaciones.test.js`	`validarClave` rechaza claves sin mayúscula, sin número o con menos de 8 caracteres y acepta `Paciente2026!`.	Lógica	—	2 ✓
G-T3 Gabriel	`src/pages/acceso/Login.test.jsx`	Con el correo y la clave de la secretaria, el login navega a `/secretaria/agenda`; con una clave incorrecta muestra la alerta y se queda en `/login`.	Componente, Interacción	— (usa la semilla real de usuarios en `localStorage`)	2 ✓
G-T4 Gabriel	`src/components/comunes/RutaProtegida.test.jsx`	Sin sesión redirige a `/login` recordando la ruta de origen; con el rol permitido muestra el contenido; con otro rol vuelve al inicio con un aviso.	Componente, Mock o spy	`vi.mock` de `auth.js`: `obtenerSesion` es un `vi.fn()`	3 ✓
S-T1 Simon	`src/utils/citas.test.js`	`generarBloques`: Daniela Morales un lunes tiene 10 bloques de 30 min, de 08:00 a 12:30; Rodrigo Sepúlveda no atiende los lunes y devuelve una lista vacía.	Lógica	—	2 ✓
S-T2 Simon	`src/utils/citas.test.js`	`puedeTransicionar` permite pasar de PENDIENTE a CONFIRMADA y bloquea pasar de ATENDIDA a CANCELADA (máquina de estados de la cita).	Lógica	—	1 ✓
S-T3 Simon	`src/components/citas/SelectorHorario.test.jsx`	Las horas ocupadas salen deshabilitadas; un clic en una libre llama a `onSeleccionar` con esa hora y uno en una ocupada no lo llama; la elegida queda con `aria-pressed`; sin horas muestra un aviso.	Componente, Interacción, Mock o spy	Spy `vi.fn()` como `onSeleccionar`	5 ✓
S-T4 Simon	`src/pages/citas/Pago.test.jsx`	Al pagar se llama una sola vez a `procesarPago` con el monto del servicio ($35.000) y la tarjeta; si responde RECHAZADO navega a `/citas/pago/error`, no crea la cita y conserva la reserva.	Interacción, Mock o spy	`vi.mock` de `pagosService` y `citasService`	1 ✓
C-T1 Catalina	`src/utils/clinica.test.js`	`calcularImc(72.4, 170)` da 25,1 y `clasificarImc(25.1)` da «Sobrepeso».	Lógica	—	1 ✓
C-T2 Catalina	`src/utils/clinica.test.js`	`filtrarServicios` con tipo PLAN y el texto «VEGANA» devuelve solo PL005; sin filtros devuelve los 18 servicios del catálogo.	Lógica	—	2 ✓
C-T3 Catalina	`src/components/clinica/ServicioFila.test.jsx`	Muestra el nombre, «50 min» y «$35.000» que recibe por props; un plan, que no tiene duración por sesión, no muestra minutos.	Componente	—	2 ✓

ID	Archivo	Comportamiento que verifica	Tipo	Mock o spy	Casos
C-T4 Catalina	`src/components/clinica/FormMedicion.test.jsx`	Con un peso de 350 kg muestra el error y no guarda; con datos válidos calcula el IMC en vivo («25,1 · Sobrepeso») y llama una vez a `onGuardar` con la medición limpia; en nutrición infantil no clasifica el IMC.	Componente · Interacción · Mock o spy	Spy `vi.fn()` como `onGuardar`	3 ✓

Los cuatro tipos quedan cubiertos

- **Lógica y validación** (6): G-T1, G-T2, S-T1, S-T2, C-T1, C-T2. Funciones puras: misma entrada, misma salida.
- **Componente** (5): G-T3, G-T4, S-T3, C-T3, C-T4. Qué se ve según las props o la sesión.
- **Interacción** (4): G-T3, S-T3, S-T4, C-T4. Escribir, hacer clic y comprobar el resultado.
- **Mock o spy** (4): G-T4, S-T3, S-T4, C-T4. Detalle en la sección 3.

Cómo se consulta el DOM

Las pruebas buscan elementos como los encontraría una persona: `getByRole("button", { name: "Ingresar" })` o `getByLabelText("Peso (kg)")`. Si una etiqueta deja de estar asociada a su campo, la prueba falla; de paso, eso protege la accesibilidad.

Los eventos se disparan con `fireEvent`, y lo asíncrono se espera con `findBy...` o `vi.waitFor`.

El resto de la suite

Además de las 12 oficiales, cada integrante probó su módulo completo. Son 369 casos más que protegen reglas de negocio, datos, servicios y pantallas:

Carpeta	Archivos	Casos	Qué cubre
`src/utils`	6	120	Reglas de negocio puras: RUT, claves, bloques horarios, Luhn, IMC, reportes.
`src/data`	6	57	CRUD sobre `localStorage`, ids correlativos, reserva en `sessionStorage`.
`src/services`	4	52	Contrato asíncrono que en EP3 cambia a la API: reglas RN, permisos y errores.
`src/pages/citas`	9	56	Asistente de 4 pasos, pago, agendas y un recorrido completo por rol.
`src/pages/clinica`	11	30	Inicio, catálogo, detalle, contacto, ficha, mis pacientes y progreso.
`src/pages/acceso y contenido`	8	23	Login, registro, perfil, panel admin, dashboard, reportes, consejos y 404.
`src/components`	16	50	Componentes reutilizables por props: tablas, filtros, formularios, selector de horario.
`src/auth y App`	2	8	Sesión simulada, roles y el enrutador completo.
Total	**62**	**396**	Incluye las 12 pruebas oficiales (27 casos).

3. Mocks y spies

Un **mock** reemplaza una dependencia real por una versión controlada: la prueba decide qué devuelve. Un **spy** registra cómo se llamó una función: cuántas veces y con qué argumentos. El criterio del equipo fue usarlos solo para aislar una dependencia (sesión, servicio, reloj, librería que jsdom no dibuja) o para verificar una llamada. La lógica pura se prueba sin mocks.

G-T4 RutaProtegida · aísla la sesión

```
vi.mock("../../auth/auth.js", () => ({ obtenerSesion: vi.fn() }));

obtenerSesion.mockReturnValue(null);
renderizarEn("/admin");
expect(screen.getByText("Login desde /admin")).toBeInTheDocument();

obtenerSesion.mockReturnValue({ id: "U001", nombre: "Administración", rol: "ADMIN" });
expect(screen.getByText("Panel privado")).toBeInTheDocument();
expect(obtenerSesion).toHaveBeenCalled();
```

`RutaProtegida` depende de quién está conectado. Con `vi.mock` el módulo `auth.js` se cambia por una versión falsa y cada caso decide la sesión con `mockReturnValue`: nadie, un ADMIN o un PACIENTE. Así se prueban los tres caminos sin pasar por el login ni tocar `localStorage`.

S-T3 SelectorHorario · espía un callback

```
const onSeleccionar = vi.fn();
render(<SelectorHorario horarios={horarios} ocupados={["08:30"]} onSeleccionar={onSeleccionar} />);
fireEvent.click(screen.getByRole("button", { name: "08:00" }));
expect(onSeleccionar).toHaveBeenCalledTimes(1);
expect(onSeleccionar).toHaveBeenCalledWith("08:00");
```

El componente no guarda la hora: avisa al padre por props. El spy `vi.fn()` registra cada llamada, así que la prueba confirma que se llamó una vez y con la hora correcta, y en otro caso que un clic en una hora ocupada no lo llamó (`not.toHaveBeenCalled()`).

S-T4 Pago · aísla el pago y las citas

```
vi.mock("../../services/pagosService.js", () => ({
  procesarPago: vi.fn(),
  reembolsarPagoSinCita: vi.fn(),
}));
procesarPago.mockResolvedValue({ estado: "RECHAZADO", pagoId: "P0006", motivo: "El banco rechazó la
tarjeta." });

expect(procesarPago).toHaveBeenCalledTimes(1);
expect(procesarPago).toHaveBeenCalledWith(
  expect.objectContaining({ monto: 35000, numero: "4000 0000 0000 0002" }));
expect(crearCita).not.toHaveBeenCalled();
```

El pago real elige el resultado según la tarjeta y guarda datos. El mock deja fijo un RECHAZADO con `mockResolvedValue` (una promesa resuelta) y permite comprobar tres cosas: que se cobró una sola vez, que se cobró el monto del servicio y que, al fallar, no se creó la cita. `citasService` también se simula para fijar las horas libres.

C-T4 FormMedicion · espía el guardado

```
const onGuardar = vi.fn().mockResolvedValue(undefined);
render(<FormMedicion onGuardar={onGuardar} objetivo="PERDIDA_PESO" hoy={HOY} />);
expect(onGuardar).toHaveBeenCalledTimes(1);
expect(onGuardar).toHaveBeenCalledWith({ fecha: "2026-10-09", pesoKg: 72.4, tallaCm: 170,
  cinturaCm: 84, caderaCm: null, grasaPct: 29.9, notas: "Control quincenal" });
```

El formulario entrega la medición a su padre mediante `onGuardar`. El spy confirma que con datos inválidos no se llama y que con datos válidos se llama una vez con los números convertidos y las notas sin espacios sobrantes. `mockResolvedValue` imita el guardado asíncrono para que el formulario muestre «Medición registrada.».

Otros dobles de prueba en la suite

Técnica	Dónde	Para qué
`vi.mock` de servicios	Dashboard, Reportes, AdminUsuarios, Inicio, MiProgreso, MisPacientes, FichaPaciente	Aislar la página de los datos y controlar la respuesta, incluido el caso de error o de lista vacía.
`vi.mock` de componentes pesados	GraficoBarras en Reportes, GraficoProgreso en MiProgreso, MapaDiferido en Inicio y Contacto, `recharts` en GraficoProgreso	jsdom no dibuja gráficos ni mapas; se prueban los datos que reciben, no el dibujo.
`vi.useFakeTimers({ toFake: ["Date"] })` y `vi.setSystemTime`	citasService, MisCitas, AgendaNutricionista, AgendaSecretaria	Fijar «hoy» en un lunes de 2030 para que las reglas de 24 horas y las horas pasadas den siempre lo mismo.
`vi.stubGlobal("matchMedia", ...)`	GraficoProgreso	jsdom no trae `matchMedia` y Recharts la consulta.
`vi.spyOn(console, "error")`	RecorridoCitas	El recorrido completo por rol no debe producir errores de React en la consola.
`vi.fn()` como callback	17 archivos de prueba	Verificar que un componente avisa a su padre (`onGuardar`, `onSeleccionar`, `onCancelar`) con los datos correctos.

`vi.clearAllMocks()` o `mockReset()` en `beforeEach` borra el historial de llamadas entre casos, para que `toHaveBeenCalledTimes(1)` cuente solo lo del caso actual.

4. Métricas de cobertura

La cobertura mide qué parte del código se ejecutó mientras corrían las pruebas. No dice si el resultado era el correcto: eso lo deciden los `expect`. Se midieron 102 archivos de `src/`.

Statements

93,51 %

2496 de 2669

Instrucciones ejecutadas al menos una vez. Una línea puede tener varias, como `const a = 1; return a;`.

Branches

83,30 %

1676 de 2012

Caminos posibles de cada decisión: las dos salidas de un `if` o de un ternario, y los atajos de `&&`, `??` y `?.`.

Functions

94,28 %

842 de 893

Funciones llamadas al menos una vez: componentes, manejadores como `onClick` y callbacks de `map`.

Lines

94,69 %

2194 de 2317

Líneas con código que se ejecutaron. Es la cifra más intuitiva, pero la menos exigente.

Cómo leer las cuatro cifras juntas

- **Lines y statements altos (94-95 %)**: casi todo el código se ejecuta al menos una vez en alguna prueba.
- **Branches más bajo (83,30 %)**: el camino feliz está probado, pero no todos los caminos alternativos. En un `if (error)` sin probar el caso con error, la línea cuenta como cubierta y la rama no. Por eso las ramas son la métrica más exigente y la más honesta.
- **Functions (94,28 %)**: casi todo se llama. De las 51 funciones sin ejecutar, 8 son de `UsuarioFormulario`; el resto son manejadores sueltos de las agendas, la carga diferida de las rutas y la tecla Escape de los diálogos.

Por dueño

Dueño	Módulo	Archivos	Al 100 %	Stmts	Branch	Funcs	Lines
Gabriel	Acceso y base común	38	13	88,92	74,86	87,86	89,86
Simon	Citas y pagos	30	6	95,10	87,80	96,09	96,29
Catalina	Sitio público y clínica	34	11	94,75	84,45	96,35	96,18
Total	Frontend completo	**102**	**30**	**93,51**	**83,30**	**94,28**	**94,69**

Gabriel incluye la base común que creó en G-00: App, Navbar, RutaProtegida, layouts, config, `almacen.js` y `formato.js`. El Footer y la información de la clínica cuentan para Catalina, según la tabla de dueños del README.

Por carpeta (salida de `npm run test:coverage`)

Carpeta	% Stmts	% Branch	% Funcs	% Lines
`src/auth`	96,55	100	100	96,29
`src/components/acceso`	85,84	67,16	88,57	86,59
`src/components/citas`	96,80	86,91	97,77	98,85
`src/components/clinica`	95,51	86,79	97,56	95,52
`src/components/comunes`	94,11	76,92	91,66	96,96
`src/config`	85,71	100	100	85,71
`src/data`	96,68	80,08	95,87	97,72

Carpeta	% Stmts	% Branch	% Funcs	% Lines
`src/layouts`	80,00	**78,57**	**66,66**	80,00
`src/pages/acceso`	98,57	86,11	100	98,50
`src/pages/acceso/admin`	**71,79**	**52,38**	**71,11**	**70,37**
`src/pages/citas`	91,26	85,75	92,21	92,51
`src/pages/citas/admin`	98,41	81,35	100	100
`src/pages/clinica`	97,24	85,06	97,29	98,26
`src/pages/clinica/admin`	89,02	**76,92**	88,88	89,18
`src/pages/contenido`	97,22	81,25	100	96,66
`src/rutas`	81,81	100	**33,33**	100
`src/services`	95,83	96,96	93,87	97,02
`src/utils`	96,11	89,35	100	98,90

En rojo, bajo 60 %; en ocre, entre 60 % y 80 %. El análisis de estos valores está en la sección 5.

5. Análisis

5.1 Lo mejor cubierto y por qué

- **`src/utils`: 98,90 % de líneas y 100 % de funciones.** Son funciones puras con las reglas del negocio: RUT, claves, bloques horarios, transiciones de estado, Luhn, IMC y filtros. No dependen de React ni del navegador, así que se prueban con entrada y salida esperada, y con muchos casos (120).
- **`src/services`: 96,96 % de ramas.** Es la capa que en EP3 hablará con la API. Se probó directamente contra los datos simulados, incluidos los errores: correo repetido, el admin desactivándose a sí mismo, una fecha pasada, un pago rechazado o un monto distinto al del servicio.
- **`src/auth`: 100 % de ramas.** La sesión decide qué ve cada rol; por eso tiene pruebas propias además de G-T4.
- **Citas y clínica sobre 96 % de líneas.** Simon sumó un recorrido completo por rol (`RecorridoCitas.test.jsx`) y Catalina probó cada una de las 11 páginas de su módulo.

5.2 Lo peor cubierto y por qué

Archivo	Lines	Branch	Por qué	Decisión
`pages/acceso/admin/UsuarioFormulario.jsx`	**0 %**	**0 %**	Ninguna prueba renderiza esta página. Su formulario, `FormUsuario`, sí está probado por separado, así que lo que queda sin probar es la orquestación: cargar el usuario, elegir entre crear o editar y volver al listado con el aviso.	Es la siguiente prueba que conviene escribir (punto 5.4).
`components/acceso/GraficoBarras.jsx`	**0 %**	**0 %**	En `Reportes.test.jsx` se reemplaza con `vi.mock`: Recharts necesita medir el contenedor y jsdom no calcula tamaños. Lo que se simula no se mide.	Aceptado. El gráfico se revisó en Chromium y Reportes sí está probado.
`config/paleta.js`	**0 %**	100 %	Solo exporta los colores del gráfico anterior; como el gráfico está simulado, nadie lo importa durante las pruebas.	Aceptado. No tiene lógica.
`data/almacen.js`	**62,50 %**	100 %	`restablecerDatosDemo()` se ejecuta a mano antes de grabar el video; ninguna prueba la llama.	Prueba corta pendiente: escribir datos, restablecer y comparar con la semilla.

Archivo	Lines	Branch	Por qué	Decisión
`layouts/LayoutAdmin.jsx`	72,72 %	62,50 %	No se ejercitan el botón «Cerrar sesión» del menú lateral ni el botón que abre ese menú en pantallas chicas.	Se probó a mano en 360 y 768 px y en el recorrido por rol.
`components/clinica/ConfirmarDesactivacion.jsx`	80 %	20 %	Se prueban los botones Confirmar y Cancelar, pero ningún caso pulsa la tecla Escape, así que `manejarTecla` nunca se ejecuta.	Agregar un caso con Escape.
`components/comunes/DialogoConfirmacion.jsx`	80 %	42,85 %	Mismo caso que el anterior: el cierre con Escape queda sin recorrer.	Agregar un caso con Escape.
`src/rutas (carpeta)`	100 %	100 %	Sus funciones están en 33,33 %. Cada página pesada se carga con `React.lazy(() => import(...))`. Esa función solo corre al visitar la ruta, y las pruebas renderizan las páginas directamente.	Aceptado. El recorrido en Chromium visita las 21 rutas.

Estos archivos explican casi todo lo que falta en `src/pages/acceso/admin` (52,38 % de ramas) y en la columna de Gabriel en la tabla por dueño.

5.3 Un error detectado gracias a una prueba

> **Reprogramar una cita bloqueaba su propio horario (RN-10).** Al reprogramar, las horas libres se calculaban contando la misma cita como ocupada. Una cita de 50 minutos a las 09:00 tapaba las 09:30, así que no se podía mover media hora más tarde aunque ese tramo iba a quedar libre.

Simon lo corrigió en el commit `bf5b39c` (16-10-2026): `horariosDisponibles` recibe un `idExcluido` y deja la propia cita fuera del cálculo. Junto con la corrección agregó la prueba que reproduce el caso.

```
export async function horariosDisponibles(nutricionistaId, fecha, duracionMin = 30, idExcluido = null) {
  ...
  const otras = citas.filter((c) => c.id !== idExcluido);
  return citasUtils.horariosDisponibles(nutricionista, fecha, otras, duracionMin, new Date());
}
```

```
it("con idExcluido, la propia cita no ocupa su horario", async () => {
  const propia = crear({ ..., nutricionistaId: "NUT003", fecha: "2030-01-07",
                         hora: "09:00", duracionMin: 50 });
  expect(await horariosDisponibles("NUT003", "2030-01-07", 50)).not.toContain("09:30");
  expect(await horariosDisponibles("NUT003", "2030-01-07", 50, propia.id)).toContain("09:30");
});
```

Para comprobar que la prueba realmente detecta el error, el 18-10-2026 se cambió esa línea por `const otras = citas;`, que es como funcionaba antes. La prueba falló:

```
FAIL  src/services/citasService.test.js > citasService: horariosDisponibles al reprogramar
      > con idExcluido, la propia cita no ocupa su horario
AssertionError: expected [ '08:00', '10:00', '10:30', …(3) ] to include '09:30'
      Tests  1 failed | 37 passed (38)
```

Al restaurar la línea, las 38 pruebas del archivo volvieron a pasar. Este procedimiento se llama **prueba de mutación**: se rompe el código a propósito y se verifica que alguna prueba lo note. La primera línea del `expect` documenta el comportamiento sin excluir y la segunda el comportamiento al reprogramar; si alguien quita el filtro en el futuro, `npm test` lo detiene antes de llegar a `main`.

5.4 La siguiente prueba que conviene

`UsuarioFormulario.test.jsx` con `vi.mock` de `usuariosService`

Es el archivo con más código sin cubrir de todo el proyecto: 26 líneas, 28 ramas y 8 funciones. Tres casos:

1. Editar un usuario y guardar: `actualizarUsuario` se llama una vez con el id de la URL y la página vuelve a `/admin/usuarios` con «Cambios de <nombre> guardados.».
2. Crear con un correo repetido: `crearUsuario` rechaza con un error de campo y el mensaje aparece bajo «Correo» sin salir de la página.
3. Abrir `/admin/usuarios/U999`: `obtenerUsuario` devuelve `null` y se muestra «El usuario que buscas no existe.».

Después vendrían Escape en los diálogos de confirmación y `restablecerDatosDemo()`, que son cortas y suben las ramas de `components/comunes` y `data`.

6. Conclusiones

- **Las 12 pruebas oficiales pasan** (27 casos) y cubren los cuatro tipos que pide la guía: lógica, componentes, interacciones y mocks o spies. La suite completa tiene 396 casos en 62 archivos, sin fallos, y corre en unos 25 segundos.
- **La cobertura es alta en tres de cuatro métricas** (93,51 % statements, 94,28 % functions, 94,69 % lines). Las ramas, con 83,30 %, muestran que faltan sobre todo caminos de error y de teclado, no funciones completas.
- **Lo que falta está identificado y tiene causa**: una página sin prueba (UsuarioFormulario), componentes que se simulan porque jsdom no dibuja y funciones de carga diferida. Cada caso tiene una decisión en la sección 5.2.

Por qué una cobertura alta no garantiza calidad

La cobertura cuenta qué se ejecutó, no qué se comprobó. Tres ejemplos de este mismo proyecto:

- **Un error que ninguna prueba vio.** En la integración del 18-10-2026, la tabla de `/admin/servicios` se salía 117 px de la pantalla a 992 px de ancho. Las pruebas corren con `css: false` en jsdom, que no calcula diseño, así que el archivo tenía cobertura y el error igual existía. Se encontró revisando las 21 rutas en Chromium a 360, 768, 992 y 1280 px.
- **Código cubierto que ninguna prueba necesitaba.** En C-08 había un filtro repetido en la página Mi progreso. Sus líneas salían cubiertas, pero al borrarlo ninguna prueba falló: era código que no aportaba nada y se eliminó. Una línea cubierta no es lo mismo que una línea verificada.
- **Código sin cubrir que funciona.** `GraficoBarras.jsx` marca 0 % porque se simula en su prueba, pero funciona y se revisó en el navegador. La cifra depende de lo que se decide simular.

Por eso, además de mirar el porcentaje, el equipo verificó con mutaciones (romper una línea y ver que una prueba falle), con el recorrido por rol en el navegador y con la revisión del diseño responsivo.

Hacia EP3

Los servicios ya devuelven promesas con el mismo contrato que tendrá la API, así que las pruebas de páginas que simulan servicios siguen sirviendo. En el backend las pruebas pasan a JUnit 5 con Mockito, con la misma idea: lógica pura sin mocks y dependencias externas simuladas.

7. Capturas

Tomadas el 18-10-2026 sobre `main` con las tres ramas integradas, la misma versión que se etiqueta como `ep2-entrega`.

Terminal · pruebas

```
~/NutriVida_Proyect/frontend $ npm test
> nutrivida-frontend@0.0.0 test
> vitest run
 Test Files  62 passed (62)
      Tests  396 passed (396)
   Start at  02:04:32
   Duration  23.43s (transform 2.61s, setup 8.98s, import 5.92s, tests 15.66s, environment 28.27s)
~/NutriVida_Proyect/frontend $ npx vitest run --reporter=verbose -t "[GSC]-T[1-4]"
 ✓ pages/citas/Pago.test.jsx > Pago (S-T4) > llama una vez a procesarPago con el monto del servicio y, si responde RECHAZADO, navega a /citas/pago/error
 ✓ pages/acceso/Login.test.jsx > Login (G-T3) > con las credenciales de la secretaria navega a /secretaria/agenda
 ✓ pages/acceso/Login.test.jsx > Login (G-T3) > con una clave incorrecta muestra la alerta y no navega
 ✓ components/clinica/FormMedicion.test.jsx > FormMedicion (C-T4) > con peso fuera de rango muestra el error y no llama a onGuardar
 ✓ components/clinica/FormMedicion.test.jsx > FormMedicion (C-T4) > con datos válidos muestra el IMC en vivo y llama a onGuardar una vez con la medición
 ✓ components/clinica/FormMedicion.test.jsx > FormMedicion (C-T4) > en nutrición infantil calcula el IMC sin clasificarlo
 ✓ components/citas/SelectorHorario.test.jsx > SelectorHorario (S-T3) > deshabilita los horarios ocupados y deja elegibles los libres
 ✓ components/citas/SelectorHorario.test.jsx > SelectorHorario (S-T3) > un clic en una hora libre llama a onSeleccionar con esa hora
 ✓ components/citas/SelectorHorario.test.jsx > SelectorHorario (S-T3) > un clic en una hora ocupada no llama a onSeleccionar
 ✓ components/citas/SelectorHorario.test.jsx > SelectorHorario (S-T3) > marca la hora seleccionada con aria-pressed
 ✓ components/citas/SelectorHorario.test.jsx > SelectorHorario (S-T3) > sin horarios muestra un aviso en vez de botones
 ✓ components/clinica/ServicioFila.test.jsx > ServicioFila (C-T3) > muestra el nombre, "50 min" y "$35.000" recibidos por props
 ✓ components/clinica/ServicioFila.test.jsx > ServicioFila (C-T3) > un plan sin duración no muestra minutos
 ✓ components/comunes/RutaProtegida.test.jsx > RutaProtegida (G-T4) > sin sesión redirige a /login recordando la ruta de origen
 ✓ components/comunes/RutaProtegida.test.jsx > RutaProtegida (G-T4) > con el rol permitido muestra el contenido protegido
 ✓ components/comunes/RutaProtegida.test.jsx > RutaProtegida (G-T4) > con otro rol no muestra el contenido y vuelve al inicio con un aviso
 ✓ utils/clinica.test.js > calcularImc y clasificarImc (C-T1) > calcularImc(72.4, 170) da 25.1 y clasificarImc(25.1) da Sobrepeso
 ✓ utils/clinica.test.js > filtrarServicios (C-T2) > con tipo PLAN y texto "VEGANA" devuelve solo PL005
 ✓ utils/clinica.test.js > filtrarServicios (C-T2) > sin filtros devuelve los 18 servicios
 ✓ utils/citas.test.js > generarBloques > S-T1: Daniela Morales un lunes da 10 bloques de 08:00 a 12:30
 ✓ utils/citas.test.js > generarBloques > S-T1: Rodrigo Sepúlveda un lunes no atiende y devuelve una lista vacía
 ✓ utils/citas.test.js > puedeTransicionar > S-T2: permite PENDIENTE a CONFIRMADA y bloquea ATENDIDA a CANCELADA
 ✓ utils/validaciones.test.js > validarRut (G-T1) > acepta un RUT con dígito verificador correcto
 ✓ utils/validaciones.test.js > validarRut (G-T1) > rechaza un RUT con dígito verificador incorrecto
 ✓ utils/validaciones.test.js > validarRut (G-T1) > acepta el RUT sin puntos ni guion y con K minúscula
 ✓ utils/validaciones.test.js > validarClave (G-T2) > rechaza claves sin mayúscula, sin número o con menos de 8 caracteres
 ✓ utils/validaciones.test.js > validarClave (G-T2) > acepta "Paciente2026!"
 Test Files  9 passed | 53 skipped (62)
      Tests  27 passed | 369 skipped (396)
```

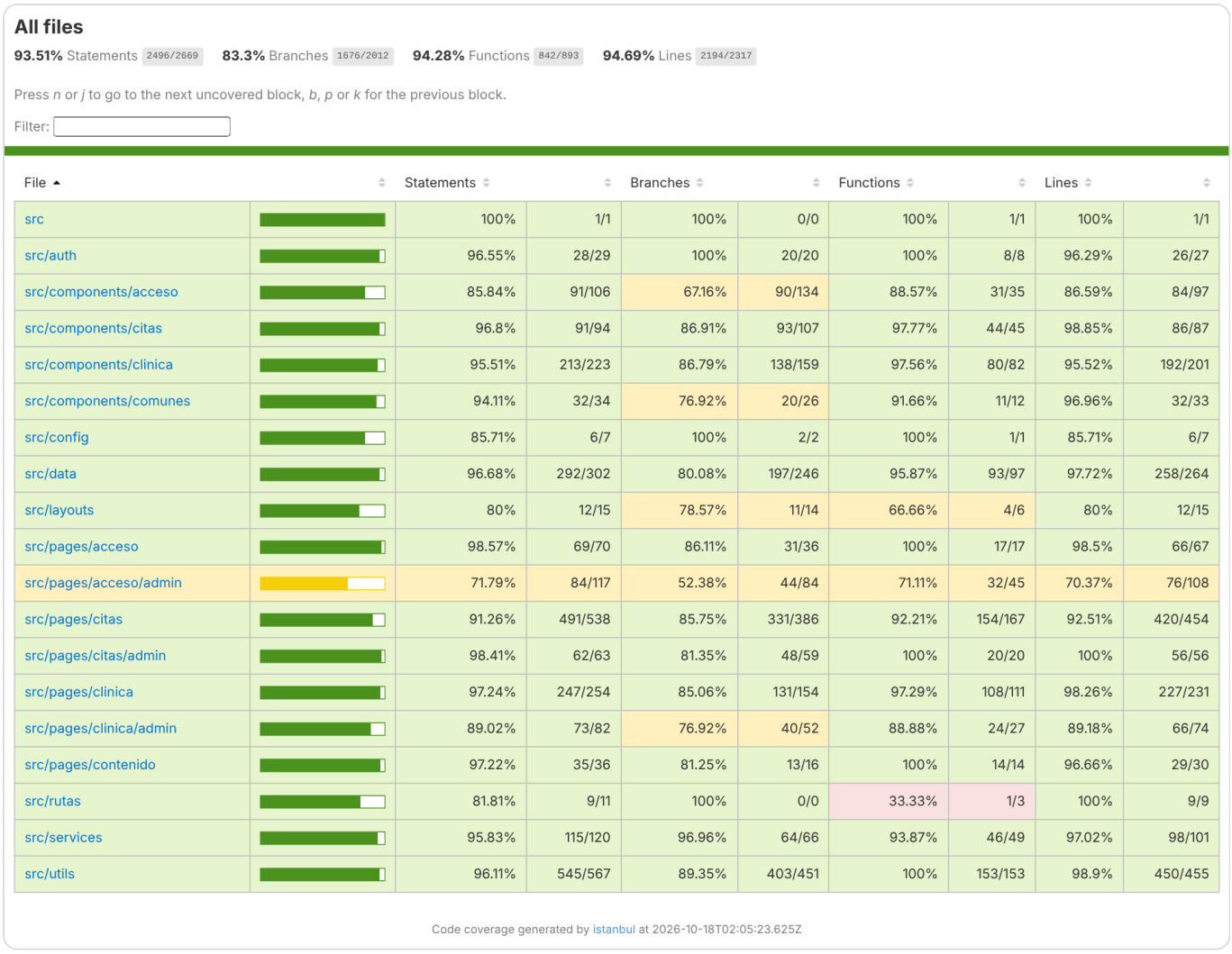
Figura 1. `npm test`: 62 archivos y 396 pruebas aprobadas. Debajo, el filtro de las 12 pruebas oficiales con sus 27 casos.

Terminal · cobertura

```
~/NutriVida_Proyect/frontend $ npm run test:coverage
------------------|---------|----------|---------|---------|-------------------
File              | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
------------------|---------|----------|---------|---------|-------------------
 src/auth         |   96.55 |      100 |     100 |   96.29 |
 ...ponents/acceso |   85.84 |    67.16 |   88.57 |   86.59 |
 ...mponents/citas |    96.8 |    86.91 |   97.77 |   98.85 |
 ...onents/clinica |   95.51 |    86.79 |   97.56 |   95.52 |
 ...onents/comunes |   94.11 |    76.92 |   91.66 |   96.96 |
 src/config       |   85.71 |      100 |     100 |   85.71 |
 src/data         |   96.68 |    80.08 |   95.87 |   97.72 |
 src/layouts      |      80 |    78.57 |   66.66 |      80 |
 src/pages/acceso |   98.57 |    86.11 |     100 |    98.5 |
 ...s/acceso/admin |   71.79 |    52.38 |   71.11 |   70.37 |
 src/pages/citas  |   91.26 |    85.75 |   92.21 |   92.51 |
 ...es/citas/admin |   98.41 |    81.35 |     100 |     100 |
 src/pages/clinica |   97.24 |    85.06 |   97.29 |   98.26 |
 .../clinica/admin |   89.02 |    76.92 |   88.88 |   89.18 |
 ...ages/contenido |   97.22 |    81.25 |     100 |   96.66 |
 src/rutas        |   81.81 |      100 |   33.33 |     100 |
 src/services     |   95.83 |    96.96 |   93.87 |   97.02 |
 src/utils        |   96.11 |    89.35 |     100 |    98.9 |
------------------|---------|----------|---------|---------|-------------------

============================== Coverage summary ==============================
Statements   : 93.51% ( 2496/2669 )
Branches     : 83.3% ( 1676/2012 )
Functions    : 94.28% ( 842/893 )
Lines        : 94.69% ( 2194/2317 )
==============================================================================
```

Figura 2. `npm run test:coverage`: tabla por carpeta y resumen de statements, branches, functions y lines.



All files

93.51% Statements 2496/2669 **83.3%** Branches 1676/2012 **94.28%** Functions 842/893 **94.69%** Lines 2194/2317

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
src	100%	1/1	100%	0/0	100%	1/1	100%	1/1
src/auth	96.55%	28/29	100%	20/20	100%	8/8	96.29%	26/27
src/components/acceso	85.84%	91/106	67.16%	90/134	88.57%	31/35	86.59%	84/97
src/components/citas	96.8%	91/94	86.91%	93/107	97.77%	44/45	98.85%	86/87
src/components/clinica	95.51%	213/223	86.79%	138/159	97.56%	80/82	95.52%	192/201
src/components/comunes	94.11%	32/34	76.92%	20/26	91.66%	11/12	96.96%	32/33
src/config	85.71%	6/7	100%	2/2	100%	1/1	85.71%	6/7
src/data	96.68%	292/302	80.08%	197/246	95.87%	93/97	97.72%	258/264
src/layouts	80%	12/15	78.57%	11/14	66.66%	4/6	80%	12/15
src/pages/acceso	98.57%	69/70	86.11%	31/36	100%	17/17	98.5%	66/67
src/pages/acceso/admin	71.79%	84/117	52.38%	44/84	71.11%	32/45	70.37%	76/108
src/pages/citas	91.26%	491/538	85.75%	331/386	92.21%	154/167	92.51%	420/454
src/pages/citas/admin	98.41%	62/63	81.35%	48/59	100%	20/20	100%	56/56
src/pages/clinica	97.24%	247/254	85.06%	131/154	97.29%	108/111	98.26%	227/231
src/pages/clinica/admin	89.02%	73/82	76.92%	40/52	88.88%	24/27	89.18%	66/74
src/pages/contenido	97.22%	35/36	81.25%	13/16	100%	14/14	96.66%	29/30
src/rutas	81.81%	9/11	100%	0/0	33.33%	1/3	100%	9/9
src/services	95.83%	115/120	96.96%	64/66	93.87%	46/49	97.02%	98/101
src/utils	96.11%	545/567	89.35%	403/451	100%	153/153	98.9%	450/455

Code coverage generated by istanbul at 2026-10-18T02:05:23.625Z

Figura 3. Informe `coverage/index.html` generado por coverage-v8. Amarillo marca entre 50 % y 80 %; rojo, bajo 50 %.